

Návrh hry Flood (Flood It)

Popis hry

Flood (Flood It) je logická dlaždicová hra pre jedného hráča. Hracie pole je mriežka vyplnená farebnými dlaždicami. Hráč začína v ľavom hornom rohu a v každom ťahu si zvolí farbu. Všetky dlaždice súvisle prepojené s ľavým horným rohom zmenia farbu na zvolenú. Cieľom je zaplaviť celé pole jednou farbou v obmedzenom počte ťahov.

Hra podporuje tri úrovne obtiažnosti s rôznymi rozmermi poľa a limitom ťahov:

Obt'ážnosť	Pole	Max ťahov
Jednoduchá	6×6	20
Stredná	8×8	25
Tazka	12×12	30

Viac informácií: [Flood-It \(Wikipedia\)](#)

Krok 2 – Konceptuálny model (Diagram tried)

Úloha 2.1 – Analýza objektov a akcií

Identifikované objekty (entity)

Objekt	Vlastnosti	Akcie
Field	rows, cols, tiles[][], maxMoves, moveCount, gameState	generate(), flood(color), isSolved(), getFloodedCount(), getBestHintColor(), getTile(r,c)
Tile	color, state	setColor(color), getColor(), getState(), setState(state)
TileColor	RED, BLUE, GREEN, YELLOW, PURPLE, ORANGE	random()
TileState	FLOODED, NOT_FLOODED	–
GameState	PLAYING, SOLVED, FAILED	–
Score	game, player, points, playedOn	getter/setter
Comment	game, player, content, commentedOn	getter/setter

Objekt	Vlastnosti	Akcie
Rating	game, player, stars, ratedOn	getter/setter
ScoreService	–	addScore(score), getTopScores(game), reset()
CommentService	–	addComment(comment), getComments(game), reset()
RatingService	–	setRating(rating), getAverageRating(game), getRating(game, player), reset()
ConsoleUI	field, playerName, difficultyName, scoreService, commentService, ratingService	play(), show(), handleInput(), selectDifficulty(), showProgress(), showHint()
Flood	–	main()

Vzt'ahy medzi objektami

- **Field** *obsahuje* (kompozícia) pole **Tile** objektov (1 : N).
- **Tile** *má* (asociácia) **TileColor** a **TileState**.
- **ConsoleUI** *používa* **Field**, **ScoreService**, **CommentService**, **RatingService**.
- **ScoreServiceJDBC**, **CommentServiceJDBC**, **RatingServiceJDBC** *implementujú* príslušné rozhrania.
- **Flood** *vytvára* **ConsoleUI** a injektuje do nej servisné komponenty.

Úloha 2.2 – Štruktúra balíkov a tried

```

sk.tuke.gamestudio/
├── entity/                                – dátové objekty (POJO)
│   ├── Score.java
│   ├── Comment.java
│   └── Rating.java
├── service/                              – servisná vrstva
│   ├── ScoreService.java                – rozhranie
│   ├── ScoreServiceJDBC.java            – JDBC implementácia
│   ├── ScoreException.java
│   ├── CommentService.java              – rozhranie
│   ├── CommentServiceJDBC.java          – JDBC implementácia
│   ├── CommentException.java
│   ├── RatingService.java               – rozhranie
│   ├── RatingServiceJDBC.java           – JDBC implementácia
│   └── RatingException.java
└── game/flood/                          – herný balík
    └── Flood.java                       – vstupný bod aplikácie (main)

```

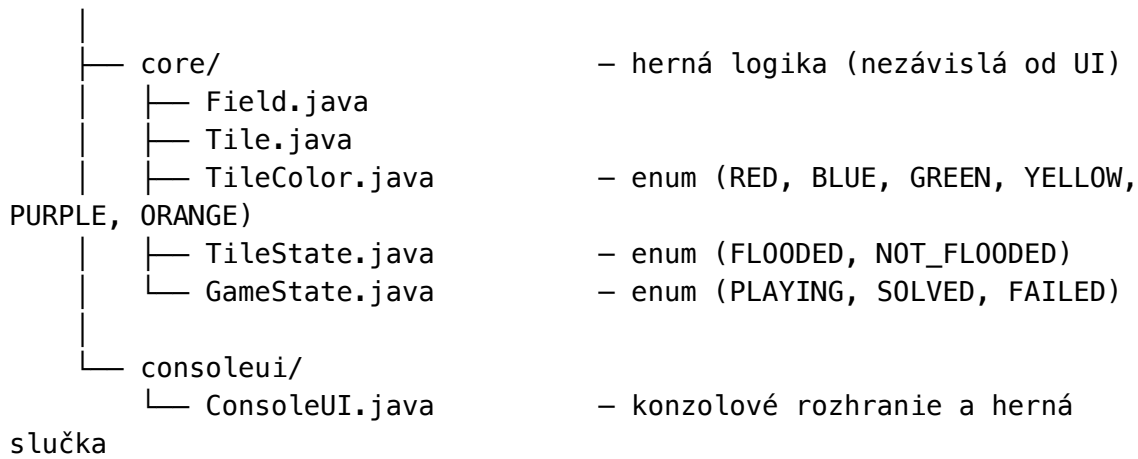
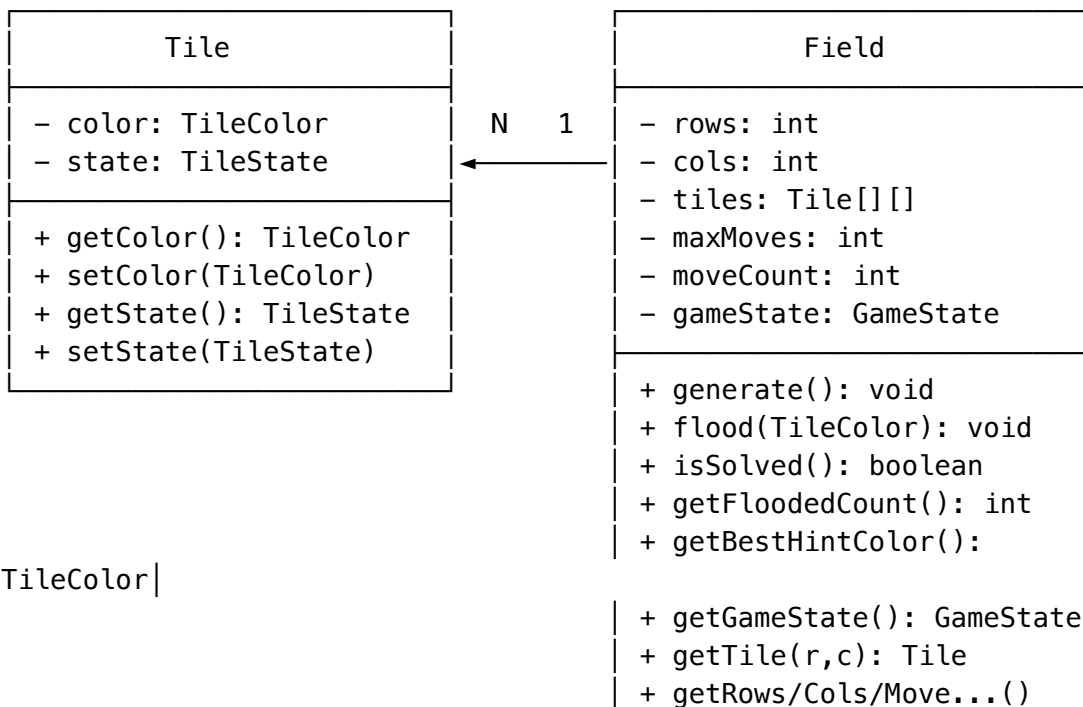
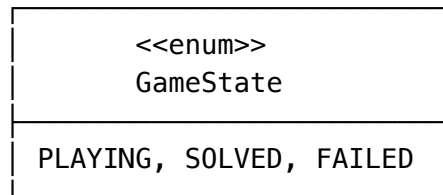
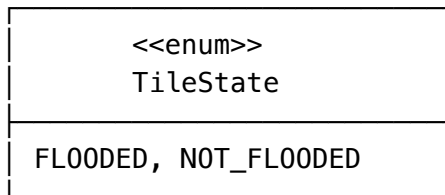
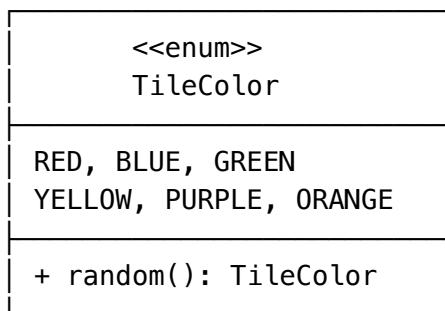
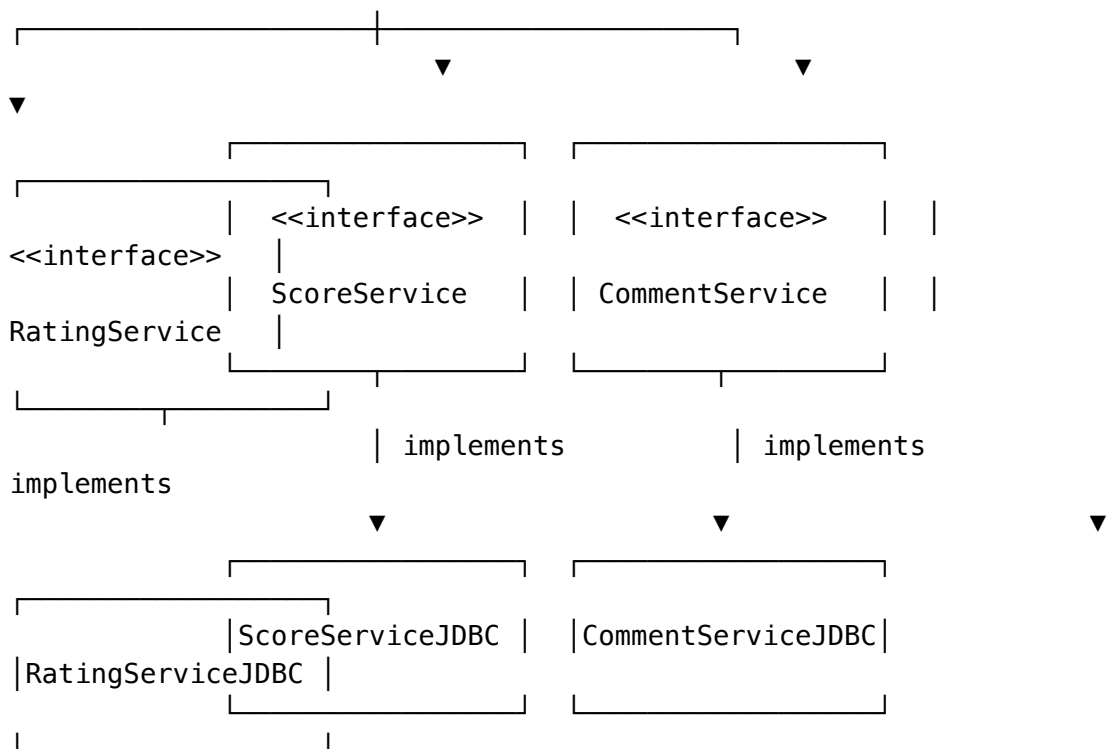
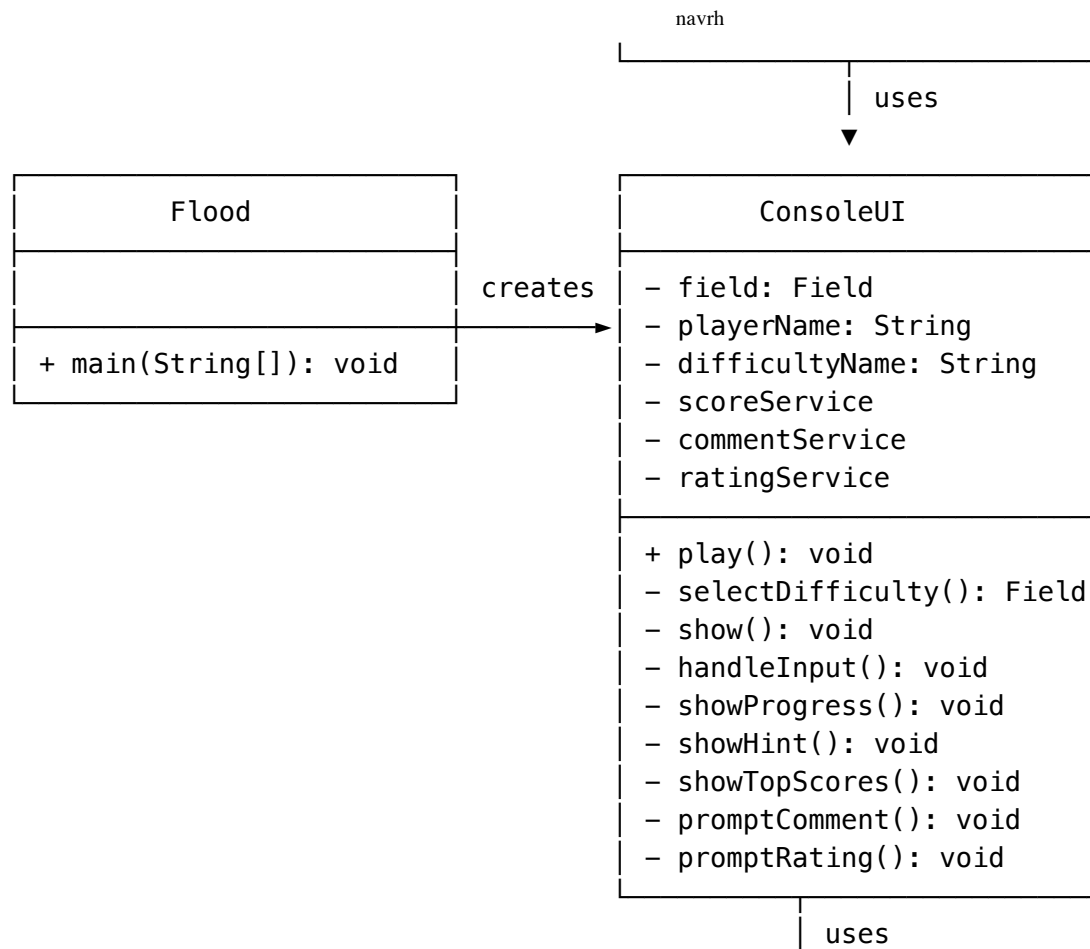


Diagram tried (textová reprezentácia UML)





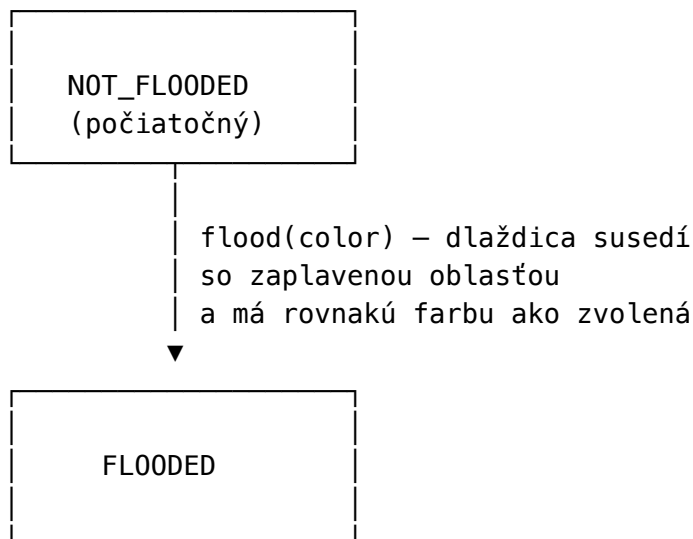
Krok 3 – Stavový diagram

Úloha 3.1 – Stavy dlaždíc

Dlaždica v hre Flood môže byť v dvoch stavoch:

Stav	Popis
NOT_FLOODED	Dlaždica ešte nie je súčasťou zaplavenej oblasti (počiatočný stav)
FLOODED	Dlaždica je súčasťou zaplavenej oblasti (prepojená s ľavým horným rohom)

Stavový diagram dlaždice (TileState):

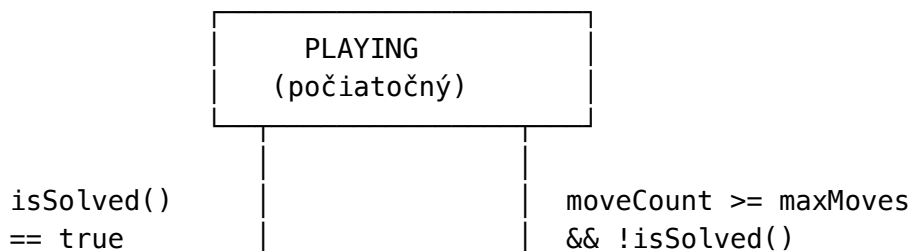


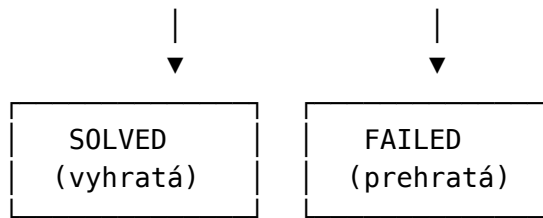
Poznámka: Prechod z NOT_FLOODED do FLOODED je jednosmerný – raz zaplavená dlaždica sa už nikdy neodplaví.

Stavy hry

Stav	Popis
PLAYING	Hra prebieha, hráč vyberá farby.
SOLVED	Celé pole je zaplavené jednou farbou – hráč vyhral.
FAILED	Hráč vyčerpal maximálny počet ťahov bez zaplavenia celého poľa.

Stavový diagram hry (GameState):



**Prechody:**

Z stavu	Do stavu	Podmienka / Akcia
PLAYING	PLAYING	flood(color) – pole nie je celé zaplavené a ostávajú ťahy
PLAYING	SOLVED	flood(color) → isSolved() == true (všetky dlaždice sú FLOODED)
PLAYING	FAILED	flood(color) → moveCount >= maxMoves a !isSolved()

Krok 4 – Základná logika hry

Úloha 4.1 – Opis hernej logiky

1. Generovanie herného poľa (Field.generate())

1. Resetovať moveCount = 0 a gameState = PLAYING.
2. Vytvoriť 2D pole dlaždíc tiles[rows][cols], každá s náhodnou farbou z TileColor.
3. Spustiť BFS od pozície (0, 0) – všetky susediace dlaždice rovnakej farby označiť ako FLOODED.

2. Ťah hráča – zaplavenie (Field.flood(TileColor color))

1. Ak gameState != PLAYING → ignorovať.
2. Ak je zvolená farba rovnaká ako aktuálna farba oblasti → ťah sa **nepočíta**.
3. Zvýšiť moveCount o 1.
4. Zmeniť farbu všetkých FLOODED dlaždíc na color.
5. BFS od všetkých FLOODED dlaždíc – absorbovať susedné NOT_FLOODED dlaždice rovnakej farby.
6. Zavolať updateGameState().

Pseudokód:

```

flood(color):
    if gameState != PLAYING: return
    if tiles[0][0].color == color: return

    moveCount++

    for each tile in tiles:
        if tile.state == FLOODED:

```

```

        tile.setColor(color)

    queue = všetky FLOODED dlaždice
    while queue is not empty:
        current = queue.dequeue()
        for each neighbor (hore, dole, vľavo, vpravo):
            if neighbor.state == NOT_FLOODED and neighbor.color ==
color:
                neighbor.setState(FLOODED)
                queue.enqueue(neighbor)

    updateGameState()

```

3. Nápoveda (Field.getBestHintColor())

Greedy algoritmus – vyberie farbu, ktorá by po zahraní rozšírila zaplavenosť najviac:

```

getBestHintColor():
    counts = prázdna mapa TileColor -> int

    for each FLOODED tile:
        for each neighbor (hore, dole, vľavo, vpravo):
            if neighbor.state == NOT_FLOODED:
                counts[neighbor.color]++

    return color s najvyšším počtom v counts

```

4. Overovanie stavu hry

```

updateGameState():
    if isSolved():      gameState = SOLVED
    if moveCount >= maxMoves and not isSolved(): gameState = FAILED

isSolved():
    for each tile in tiles:
        if tile.state == NOT_FLOODED: return false
    return true

```

5. Herná slučka (ConsoleUI.play())

```

play():
    field = selectDifficulty()      // hráč zvolí obťažnosť (1-3)
    playerName = readPlayerName()

    field.generate()
    do:
        show()                      // výpis farebného herného poľa
        showProgress()              // progress bar: [■■■■■■■■■■] 52%
(34/64)
        print("Tah: X/Y")
        handleInput()               // R/B/G/Y/P/O = ťah, ? =

```

```
nápoveda, X = koniec
    while field.gameState == PLAYING

        show()
        printResult()                // Gratulujeme / Prehra

        showTopScores()              // TOP 10 pre danú obťažnosť
        showAverageRating()
        promptComment()
        promptRating()

    if playAgain: play()
```

6. Zobrazenie herného poľa (ConsoleUI.show())

Každá dlaždica sa zobrazí ako farebný blok s ANSI escape kódmi (farebné pozadie terminálu). Farby sú mapované na znaky aj farby pozadia:

TileColor	Znak	ANSI pozadie
RED	R	\033[41m
BLUE	B	\033[44m
GREEN	G	\033[42m
YELLOW	Y	\033[43m
PURPLE	P	\033[45m
ORANGE	O	\033[48;5;208m

Príklad výstupu (8×8 pole, Stredna obt'aznosť):

	1	2	3	4	5	6	7	8
1	[R]	[R]	[B]	[G]	[Y]	[P]	[O]	[R]
2	[B]	[R]	[G]	[G]	[Y]	[O]	[O]	[B]
...								

Zaplavene: [] 40% (26/64)

Tah: 8/25

[R] [B] [G] [Y] [P] [0]

Zvolte farbu (R/B/G/Y/P/O), ? napoveda, X ukoncenie:

7. Servisné komponenty

Skóre, komentáre a hodnotenia sú ukladané do PostgreSQL databázy cez JDBC.

- **Skóre** je viazané na kombináciu hry a obtiažnosti ("flood-jednoduchá", "flood-stredná", "flood-tazka"), aby mal každý level vlastný rebríček. Body sa počítajú ako $\text{maxMoves} - \text{moveCount}$ (menej ťahov = viac bodov).
- **Komentáre a hodnotenia** sú zdieľané pre celú hru ("flood"), nezávisle od obtiažnosti.

Databázová schéma:


```
CREATE TABLE score (  
  id      SERIAL PRIMARY KEY,  
  game    VARCHAR(64) NOT NULL,  
  player  VARCHAR(64) NOT NULL,  
  points  INTEGER     NOT NULL,  
  playedon TIMESTAMP  NOT NULL  
);  
  
CREATE TABLE comment (  
  id      SERIAL PRIMARY KEY,  
  game    VARCHAR(64)  NOT NULL,  
  player  VARCHAR(64)  NOT NULL,  
  content VARCHAR(1024) NOT NULL,  
  commentedon TIMESTAMP NOT NULL  
);  
  
CREATE TABLE rating (  
  id      SERIAL PRIMARY KEY,  
  game    VARCHAR(64) NOT NULL,  
  player  VARCHAR(64) NOT NULL,  
  stars   INTEGER     NOT NULL CHECK (stars BETWEEN 1 AND 5),  
  ratedon TIMESTAMP  NOT NULL,  
  UNIQUE (game, player)  
);
```
